

What is Abstraction as a Guideline

Abstraction is a guideline in object-oriented design that says:

- Show only what is essential, and hide the details of how it works.
 - It helps us design systems where users of a class only care about what the object can do, not how it does it internally.
 - The goal is to reduce complexity and improve flexibility by exposing only the necessary features.
-

How We Can Implement This:

1. Through Interfaces

- Interfaces define *contracts* (methods/properties that must be implemented).
- They provide abstraction by ensuring that different classes expose the same behavior without revealing the implementation.
- Example studied: IVehicle with StartEngine() and StopEngine() defines "what" a vehicle does, not "how".

2. Through Abstract Classes

- Abstract classes allow us to define abstract methods (no implementation) and concrete methods (with implementation).
- This enforces a shared structure across subclasses while still leaving details hidden.
- Example studied: Shape with GetArea() abstract method ensures every shape defines area but hides how.

3. Through Encapsulation

- Encapsulation works hand-in-hand with abstraction.
- By making fields private and exposing only necessary properties/methods, we hide implementation details.
- Example studied: Account struct with private fields and public properties shows how encapsulation supports abstraction.